Factorizing large integers

Classical, quantum, "hybrid", quantum-inspired?

Antoine Tilloy

RSA-330bits

1 522 605 027 922 533 360 535 618 378 132 637 429 718 068 114 961 380 688 657 908 494
580 122 963 258 952 897 654 000 350 692 006 139

$=$

37 975 227 936 943 673 922 808 872 755 445 627 854 565 536 638 199

$\times$

40 094 690 950 920 881 030 683 735 292 761 468 389 214 899 724 061

Quantum Information Seminar
March 2nd, 2021



Why

- Factoring is the most impressive application of quantum algorithms
- It's also weirdly peculiar
- Ewin Tang dequantized recommendation systems, factoring probably cannot be dequantized but why?
- Zhou Soutenmires and Waintal simulated quantum circuits believed to be in the supremacy regime with tensor networks, but probably can't Shor

$\Longrightarrow$ **What is going on with factoring?**

Outline

1. The problem
2. Classical factoring
3. Quantum factoring
4. Relaxed semi quantum factoring(?)

The problem

Factorization problem

Let N be a large number (n bits, with $n \gtrsim 100$), write it

$$N = p_1 \times p_2 \times \cdots \times p_k$$

where p_j are prime numbers.

- Believed not to be NP-complete (otherwise co-NP=NP)
- Believed not to be in P
- But primality testing is provably efficient (Agrawal–Kayal–Saxena 2002)

Example

RSA-829bits $=$
2 140 324 650 240 744 961 264 423 072 839 333 563 008 614 715 144 755 017 797 754 920 881 418 023 447 140 136 643 345 519 095 804 679 610 992 851 872 470 914 587 687 396 261 921 557 363 047 454 770 520 805 119 056 493 106 687 691 590 019 759 405 693 457 452 230 589 325 976 697 471 681 738 069 364 894 699 871 578 494 975 937 497 937

can be factorized (with 2600 core.years) into:

RSA-829bits $=$
64 135 289 477 071 580 278 790 190 170 577 389 084 825 014 742 943 447 208 116 859 632 024 532 344 630 238 623 598 752 668 347 708 737 661 925 585 694 639 798 853 367
$\times$
33 372 027 594 978 156 556 226 010 605 355 114 227 940 760 344 767 554 666 784 520 987 023 841 729 210 037 080 257 448 673 296 881 877 565 718 986 258 036 932 062 711

Example

RSA-2048bits = 2 519 590 847 565 789 349 402 718 324 004 839 857 142 928
212 620 403 202 777 713 783 604 366 202 070 759 555 626 401 852 588 078
440 691 829 064 124 951 508 218 929 855 914 917 618 450 280 848 912 007
284 499 268 739 280 728 777 673 597 141 834 727 026 189 637 501 497 182
469 116 507 761 337 985 909 570 009 733 045 974 880 842 840 179 742 910
064 245 869 181 719 511 874 612 151 517 265 463 228 221 686 998 754 918
242 243 363 725 908 514 186 546 204 357 679 842 338 718 477 444 792 073
993 423 658 482 382 428 119 816 381 501 067 481 045 166 037 730 605 620
161 967 625 613 384 414 360 383 390 441 492 634 432 190 114 657 544 454
178 424 020 924 616 515 723 350 778 707 749 817 125 772 467 962 926 386
356 373 289 912 154 831 438 167 899 885 040 445 364 023 527 381 951 378
636 564 391 212 010 397 122 822 120 720 357

Exponential classical methods

Naively, factorizing large numbers looks like the typical NP complete problem solvable only by enumeration

1. Bruteforce $\sqrt{N}$
2. ϵ-smart bruteforce, trying only prime numbers $\implies \sqrt{N}/\log(N)$ because

$$p_k \sim k\log(k) \quad \text{[Hadamard and de la Vallée Poussin 1896]}$$

   there are a lot of prime numbers! (k prime with proba $1/\log(k)$)
3. Pollard-Strassen method $N^{1/4+o(1)}$ rigorous deterministic
4. Shanks method $N^{1/5+o(1)}$ rigorous deterministic **if** Extended Riemann Hypothesis (ERH) is true

Subexponential classical methods

- Quadratic Sieve (QS) $\exp\left[\sqrt{\log(N)\log\log(N)}\right] = o(N^{\alpha})$ for all $\alpha > 0$
- General Number Field Sieve (GNFS) $\exp\left[C\log(N)^{1/3}\log\log(N)^{2/3}\right]$
  $\rightarrow$ faster known classical heuristic
- There is a rigorous proof that one can reach $\exp\left[\sqrt{\log(N)\log\log(N)}\right]$ with a probabilistic algorithm without ERH

Factorizing with non-trivial square roots of unity

Imagine we know $N = p \times q,\ p \neq q$ but we don't know p, q

Imagine we find **a non-trivial root of unity modulo** N, i.e. $x \in \mathbb{N}$ such that

- $x^2 \equiv 1 \bmod N$
- $x \not\equiv \pm 1 \bmod N$

Then $(x-1)(x+1) = bN$ for some b.

Since $x \not\equiv \pm 1 \bmod N$, $gcd(N, x \pm 1) \neq N$.

Hence $gcd(N, x \pm 1) = p$ or q. Importantly $gcd(a, b)$ is efficient with Euclide's algorithm

Quadratic Sieve

Shor's algorithm: number theoretic part

Let $x \in [1, 2N]$, imagine you can find r such that $x^r \equiv 1 \bmod N$
[a.k.a.: you know how to compute discrete logarithms]

Shor's key insight

Assume

1. r is even (then $x^{r/2}$ is a square root of unity)
2. $x^{r/2} \not\equiv \pm 1 \bmod N$ (this root is further non-trivial)

Then we can factor N as in the quadratic sieve by taking $gcd(N, x^{r/2} \pm 1)$

- Typically, r is going to be even half the time if we're lucky
- Typically, the root $x^{r/2}$ is going to be non-trivial half the time, since there are 4 roots and 2 trivial ones

If we have a black box to compute discrete logarithms, we should find a factor of N every 4 calls

Shor's algorithm: quantum part

Idea: amplitude amplification

Imagine you are given a unitary U_x doing modular exponentiation of x:

$$U_x|r\rangle \otimes |0\rangle := |r\rangle \otimes |x^r \bmod N\rangle$$

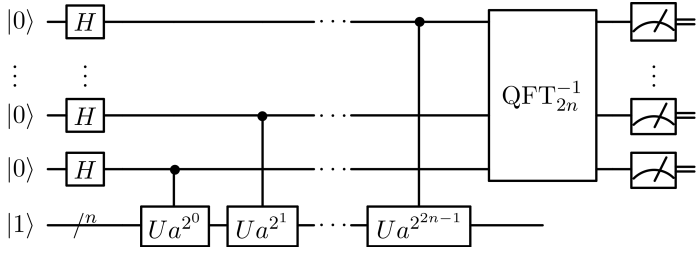
Start in uniform superposition of $|r\rangle \otimes |0\rangle$, apply U_x, then Quantum Fourier Transform (QFT) the state

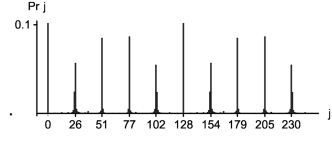
$$|\Psi\rangle = \frac{1}{Q}\sum_{k,r} e^{2i\pi kr/Q}|k\rangle \otimes |x^r \bmod N\rangle \quad Q = 2^q, \text{with } q \text{ the number of bits}$$

and measure both registers to get k, y with probability

$$\text{Prob}[k,y] = \frac{1}{Q^2}\Big|\sum_{r,x^r\equiv y} e^{2i\pi kr/N}\Big|^2 = \frac{1}{Q^2}\Big|\sum_{b=0}^{m-1} e^{2i\pi k(r(y)+r_0 b)/Q}\Big|^2 = \frac{1}{Q^2}\frac{\sin^2(\pi r_0 mk/Q)}{sin^2(\pi r_0 k/Q)}$$

Shor's quantum part, summary





Modular exponentiation

Importantly, modular exponentiation is implementable without exponential depth, by decomposing into modular multiplications

$$U|y_1\rangle \otimes |y_2\rangle := |y_1\rangle \otimes |y_1 y_2 \bmod N\rangle$$

Then one can efficiently implement the controlled r modular power

$$\mathcal{U}|r\rangle \otimes |x\rangle := |r\rangle \otimes |x^r \bmod N\rangle$$

This is by far the costliest part of the algorithm and where implementations differ (QFT costs peanuts, and is linear for fixed error)

Various cost estimates

from "Investigating the practical implementation of Shor's algorithm" Simon J. Devitt, Austin G. Fowler, Lloyd C. L. Hollenberg 2005

Circuit	Qubits	Depth
Simplicity[6]	$\sim 5L$	$O(L^3)$
Speed[7]	$O(L^2)$	$O(L \log L)$
Qubits[8]	$\sim 2L$	$\sim 32L^3$
Tradeoff I[9]	$\sim 50L$	$\sim 2^{19}L^{1.2}$
Tradeoff II[9]	$\sim 5L$	$\sim 3000L^2$

L	p_R	$k(c=10^4)$	$Q(c=10^4)$	$k(c=10^3)$	$Q(c=10^3)$	$k(c=10^2)$	$Q(c=10^2)$
64	9.3×10^{-9}	3	45,276	2	6,468	2	6,468
128	5.8×10^{-10}	3	89,180	2	12,740	2	12,740
256	3.6×10^{-11}	3	176,988	3	176,988	2	25,284
512	2.3×10^{-12}	4	2,468,228	3	352,604	2	50,372
1024	1.4×10^{-13}	4	4,926,852	3	703,836	2	100,548

Table 3. Table showing QEC requirements for the QPF subroutine. L denotes the binary length of the number to be factored. p_R is the required error rate on each logical qubit $p_R = 1/64L^4$. k is the number of levels of concatenated correction. p_{logical} is the actual logical error rate for k levels of concatenated QEC using the scaling relationship $p_{\text{logical}} = (cp)^{2^k}/c$ with $p = 10^{-5}$ and $\{c = 10^4, c = 10^3, c = 10^2\}$. Q is the minimum number of qubits required, $Q = (2L + 4)7^k$.

Factoring large biprimes on NISQ?: classical part

Starting (classical) idea from Christopher Burges, Microsoft Research, 2003

Factoring as a satisfaction problem

Imagine $N = x \times y$ and write N, x, y in binary and write the multiplication

1	0	1	1	0	1	1
			x_1	x_2	x_3	1
			y_1	y_2	y_3	1
			x_1	x_2	x_3	1
		y_3x_1	y_3x_2	y_3x_3	y_3	0
	y_2x_1	y_2x_2	y_2x_3	y_2	0	0
y_1x_1	y_1x_2	y_1x_3	y_1	0	0	0

$\Longrightarrow$

$$
\begin{aligned}
x_3 + y_3 &= 1 \\
x_2 + y_3x_3 + y_2 &= 2z_1 \\
x_1 + y_3x_2 + y_2x_3 + y_1 + z_1 &= 1 + 2z_2 + 4z_3 \\
y_3x_1 + y_2x_2 + y_1x_3 + z_2 &= 1 + 2z_4 \\
y_2x_1 + y_1x_2 + z_3 + z_4 &= 2z_5 + 4z_6 \\
y_1x_1 + z_5 &= 1 \\
z_6 &= 0
\end{aligned}
$$

The write each equation in the form $C_i = 0$ and minimize

$$H = \sum_i C_i^2$$

One can even optimize over the reals by adding minimizing
$H_{\mathbb{R}} = H + \sum x_i^2(1 - x_i)^2 + y_i^2(1 - y_i)^2 + z_i^2(1 - z_i)^2$

Factoring large biprimes on NISQ?: quantum part

Zapata (Eric R. Anschuetz, Jonathan P. Olson, Alán Aspuru-Guzik, Yudong Cao)
What if we try to minimize H with QAOA?

Variational Quantum Factoring

Eric R. Anschuetz,* Jonathan P. Olson,† Alán Aspuru-Guzik,‡ and Yudong Cao§
Zapata Computing Inc., 501 Massachusetts Avenue, Cambridge MA 02138

Abstract

Integer factorization has been one of the cornerstone applications of the field of quantum computing since the discovery of an efficient algorithm for factoring by Peter Shor. Unfortunately, factoring via Shor's algorithm is well beyond the capabilities of today's noisy intermediate-scale quantum (NISQ) devices. In this work, we revisit the problem of factoring, developing an alternative to Shor's algorithm, which employs established techniques to map the factoring problem to the ground state of an Ising Hamiltonian. The proposed variational quantum factoring (VQF) algorithm starts by simplifying equations over Boolean variables in a preprocessing step to reduce the number of qubits needed for the Hamiltonian. Then, it seeks an approximate ground state of the resulting Ising Hamiltonian by training variational circuits using the quantum approximate optimization algorithm (QAOA). We benchmark the VQF algorithm on various instances of factoring and present numerical results on its performance.

Criticism

Mostly borrowed from Aaronson's "Quantum computing motte and baileys"
https://www.scottaaronson.com/blog/?p=4447

- Sure it could run on NISQ, but why would you do it?
- This minimization encoding turns factoring into a problem that is generically NP-hard
- Not clear why QAOA would do better than classical heuristics (*e.g.* simulated annealing)
- Even if there is a poly gain (unlikely), it would never beat the Quadratic Sieve that is subexponential

Strategies?

We need to use at least some number-theoretic structure and not start from a NP-hard-looking formulation

- Relaxing from Shor's algorithm? Variationalizing Shor? Adiabatizing?
- Direct MPS approx simulation fails from volume law, is there a smart structure for sampling semi-efficiently?
- Any way to stabilize the interference despite errors? Can we put the "existence of interference" in the cost function?